

REPORTE TAREA 1

ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Análisis experimental de algoritmos de ordenamiento y multiplicación de matrices.»

Diego Ariel Araneda Campusano

8 de septiembre de 2026

00:44

1. Introducción

En el análisis y diseño de algoritmos, comparar experimentalmente las cotas de complejidad teórica con el rendimiento empírico es fundamental para comprender el impacto real de las constantes ocultas y la sobrecarga computacional [2, 4]. Este informe aborda el estudio comparativo de dos problemas computacionales clásicos: primero el ordenamiento de arreglos unidimensionales empleando *Merge Sort*, *Quick Sort* (optimizado con partición de Hoare [3] y eliminación por recursión de cola), *Patience Sort* [5, 6] y el algoritmo *Sort default*. Además la multiplicación de matrices cuadradas comparando el método clásico $\mathcal{O}(N^3)$ frente al método de partición en bloques de Strassen $\mathcal{O}(N^{2,807})$ [2, 1]. El objetivo central es contrastar las curvas empíricas de ejecución frente a sus órdenes asintóticos ideales bajo diversas familias de entradas y orden de magnitud de datos.

2. Entorno experimental

2.1. Entorno Experimental y Plataforma de Ejecución

Las pruebas de rendimiento y recolección de métricas se ejecutaron bajo condiciones de carga controladas en un equipo portátil con las siguientes especificaciones técnicas:

- **Sistema Operativo:** Zorin OS 18.1
- **Procesador:** Intel (R) Core(TM) i5-5350U CPU @ 1.80GHz
- **Memoria RAM:** 8GB

- **Compilador:** g++ (Ubuntu 13.3.0-6ubuntu2 24.04.1) 13.3.0
- **Entorno de Automatización y Análisis:** Python 3.12 utilizando las bibliotecas NumPy, Pandas y Matplotlib para la estructuración de datos y generación de gráficos.
- **Medición Temporal:** Reloj de la biblioteca estándar `std::chrono::high_resolution_clock`.

2.2. Conjuntos de Datos (Datasets)

Siguiendo las pautas del enunciado para asegurar que los experimentos sean replicables [4]:

1. **Módulo de Ordenamiento:** Se probaron arreglos de enteros con tamaños $N \in \{10^1, 10^2, 10^3, 10^4, 10^5, 10^6, 10^7\}$. Para cada tamaño, se generaron distintas configuraciones de entrada (aleatorio, ordenado, inversamente ordenado, semiordenado al 50% y con elementos repetidos) variando el rango de los números (D_1 y D_7), tomando 3 repeticiones (a, b, c) por cada caso para promediar los tiempos.
2. **Módulo de Multiplicación Matricial:** Se generaron pares de matrices cuadradas de $N \times N$ con dimensiones $N \in \{16, 64, 256, 1024\}$. Se consideraron tres tipos de matriz (densa, diagonal y dispersa con 10% de elementos no nulos) y dos rangos de valores para sus elementos ($D_0 \in \{0, 1\}$ y $D_{10} \in \{0, \dots, 9\}$), ejecutando también 3 repeticiones por combinación.

3. Resultados y Análisis

En esta sección se presentan y analizan los resultados empíricos obtenidos en las pruebas de rendimiento. Cada punto graficado corresponde al promedio aritmético de las tres réplicas independientes (a, b, c) ejecutadas para cada configuración experimental.

3.1. Resultados en Algoritmos de Ordenamiento

El módulo de ordenamiento evalúa el tiempo de ejecución en función del tamaño del arreglo $N \in [10^1, 10^7]$ combinando distintas distribuciones iniciales y dominios numéricos.

Para apreciar resultados se muestran las tres distribuciones evaluadas bajo el dominio amplio D_7 (aleatorio, descendente y ascendente). El motivo de esta decisión es debido a que: en D_7 los números casi no se repiten, lo que evita atajos accidentales por valores duplicados y permite aislar los comportamientos más interesantes y realistas del experimento:

- Algoritmos como *Merge Sort* y `std::sort` son prácticamente insensibles a la forma de la entrada: *Merge Sort* siempre divide el arreglo en mitades iguales ejecutando un costo constante de división y mezcla, y asumimos que `std::sort` es sumamente robusto debido a las optimizaciones internas de la biblioteca estándar de C++. Ambos mantienen siempre su tendencia esperada $\mathcal{O}(N \log N)$.
- Pero notamos que, *Quick Sort* y *Patience Sort* son los métodos que exhiben variaciones frente al orden inicial de los datos, justificando un análisis detallado de sus casos extremos.

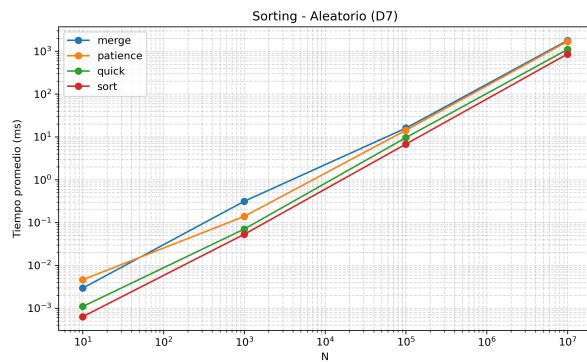


Figura 1: Caso promedio: entrada aleatoria (D_7).

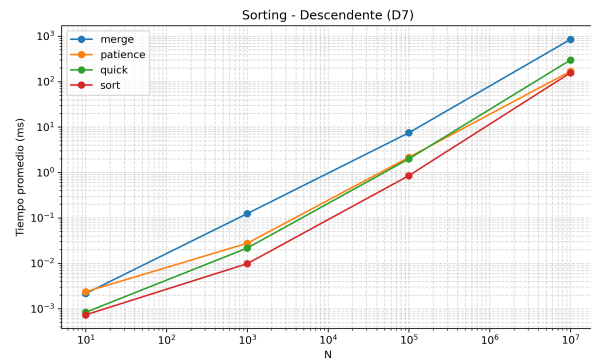


Figura 2: Mejor caso de Patience: descendente (D_7).

En la Figura 1 se presenta el comportamiento habitual con números desordenados. Como ambos ejes están en escala logarítmica (potencias de 10), una curva que se ve recta representa un crecimiento proporcional a su cota asintótica casi lineal $\mathcal{O}(N \log N)$. Podemos observar que:

1. *std::sort* y *Quick Sort* lideran en velocidad. *Quick Sort* implementa la partición de Hoare [3] (con dos punteros que recorren desde los extremos) y elimina la recursión de cola (usando un bucle para la mitad más grande), logrando tiempos muy cercanos a la biblioteca estándar de c++.
2. *Merge Sort* resulta más lento que los dos anteriores debido a la sobrecarga práctica de memoria: en cada paso recursivo necesita pedir memoria al sistema y copiar elementos a un arreglo auxiliar.
3. *Patience Sort* muestra tiempos mayores porque en una secuencia aleatoria se generan muchas pilas intermedias, exigiendo un esfuerzo continuo para insertar y extraer datos de su cola de prioridad.

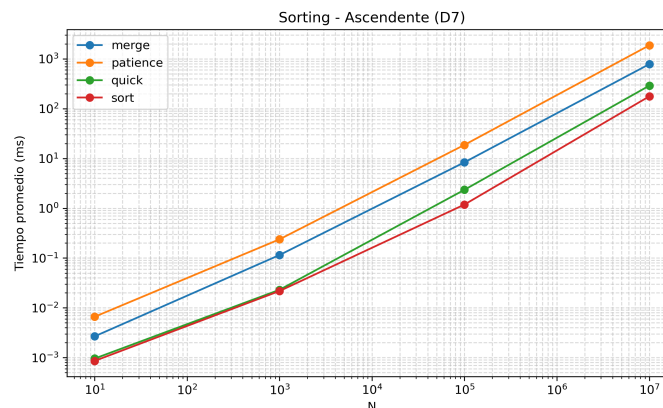


Figura 3: Peor caso de Patience: ascendente (D_7).

El contraste más revelador del experimento aparece al comparar las entradas ordenadas (Figuras 2 y 3):

- **La estabilidad de Quick Sort con esquema Hoare:** En la teoría clásica, cuando un arreglo ya está ordenado de forma ascendente o descendente, Quick Sort corre el riesgo de degradarse a un tiempo

cuadrático $\mathcal{O}(N^2)$ si elige como pivote el primer o último elemento (pues divide el arreglo en un subarreglo de $N-1$ y otro vacío). En nuestras pruebas, la curva se mantiene completamente estable y rápida. Esto confirma el éxito de seleccionar el pivote en el punto medio del arreglo y usar el esquema de Hoare, balanceando las particiones evitando una ejecución que podría tardar incluso horas.

- **La notable asimetría de Patience Sort:** Este algoritmo funciona como el juego solitario de cartas: coloca cada nuevo número sobre la primera pila cuyo tope sea mayor o igual a él.
 - En la entrada **descendente** (Figura 2), cada nuevo número que llega es menor que el anterior, por lo que todos los elementos caen sucesivamente sobre una única pila ($k = 1$). La etapa final de fusión con la cola de prioridad solo tiene que vaciar esa sola pila de principio a fin, logrando un tiempo casi lineal $\mathcal{O}(N)$ (su mejor caso posible).
 - En la entrada **ascendente** (Figura 3), cada nuevo elemento es más grande que todos los anteriores, por lo que nunca puede apilarse sobre ninguno y obliga a abrir una pila nueva por cada dato ($k = N$). La fase de reconstrucción tiene que gestionar un montículo de tamaño N , disparando el tiempo de ejecución hasta convertir a Patience Sort en el algoritmo más lento del benchmark para $N = 10^7$.

3.2. Resultados en Multiplicación Matricial

Para la multiplicación de matrices cuadradas de $N \times N$, se comparó el algoritmo tradicional *Naive* $\mathcal{O}(N^3)$ frente al método recursivo de *Strassen* $\mathcal{O}(N^{2.807})$ [1] para dimensiones $N \in \{16, 64, 256, 1024\}$ sobre dominios binarios (D_0) y enteros (D_{10}).

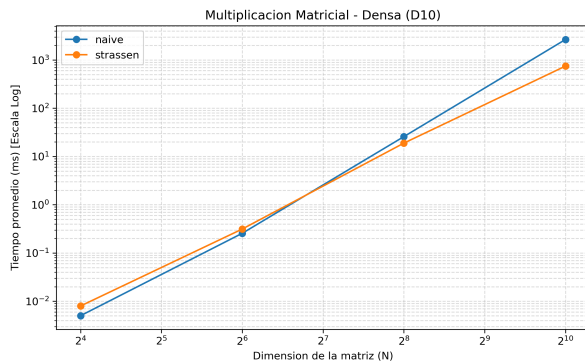


Figura 4: Multiplicación en matrices densas (D_{10}).

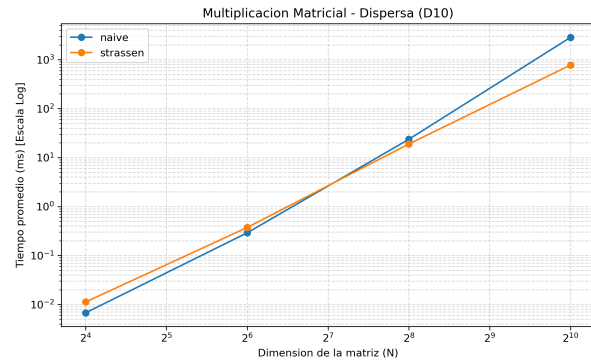


Figura 5: Multiplicación en matrices dispersas (D_{10}).

En la Figura 4 se observa claramente el compromiso entre constantes ocultas y cota asintótica:

1. **Matrices pequeñas ($N \leq 64$):** El algoritmo clásico *Naive* supera en velocidad a Strassen. Esto ocurre porque la sobrecarga de crear submatrices temporales, realizar sumas matriciales y gestionar la recursión es mayor que el ahorro que ofrece calcular 7 multiplicaciones en lugar de 8.

2. **Cruce asintótico** ($N \geq 256$): Al crecer la dimensión de las matrices, el término cúbico N^3 domina rápidamente. La menor pendiente asintótica de Strassen ($k \approx 2,81$) compensa la sobrecarga de memoria y reduce drásticamente el tiempo de ejecución para $N = 1024$.

Cuadro 1: Resumen comparativo de complejidad teórica y comportamiento empírico.

Algoritmo	Complejidad Temporal	Complejidad Espacial	Sensibilidad a la Entrada
Merge Sort	$\Theta(N \log N)$	$\Theta(N)$	Invariable ante el orden inicial
Quick Sort (Hoare)	$\mathcal{O}(N \log N)$ prom.	$\mathcal{O}(\log N)$	Robusto con pivote central
Patience Sort	$\mathcal{O}(N \log k)$	$\mathcal{O}(N)$	Extrema (depende de la subsecuencia)
std::sort	$\mathcal{O}(N \log N)$	$\mathcal{O}(\log N)$	Invariable
Naive Matrix	$\mathcal{O}(N^3)$	$\Theta(1)$	Invariable ante ceros o densidad
Strassen	$\mathcal{O}(N^{2,807})$	$\Theta(N^2)$	Invariable ante ceros o densidad

Por último, al comparar la Figura 4 con la Figura 5 (matrices dispersas al 10%) y los casos de matrices diagonales, los tiempos de ejecución de ambos métodos son casi idénticos. Esto se debe a que las implementaciones utilizan una estructura de matriz densa estándar `std::vector<std::vector<int>`, la cual itera y computa todas las posiciones sin realizar optimizaciones condicionales por la presencia de ceros.

4. Conclusiones

Concluimos que la complejidad teórica describe el crecimiento ante instancias masivas, pero las constantes ocultas y la gestión de memoria determinan el rendimiento práctico. La robustez de `std::sort` y la estabilidad de *Quick Sort* (con partición de Hoare y pivote central) demuestran cómo optimizar punteros y llamadas en la pila supera a algoritmos como *Merge Sort*, penalizado por su memoria auxiliar $\Theta(N)$. A su vez, la asimetría de *Patience Sort* ratifica que el rendimiento real oscila entre un tiempo casi lineal $\mathcal{O}(N)$ y sufre una degradación severa según el orden de entrada. En matrices, el método de *Strassen* confirmó empíricamente su menor orden asintótico frente al crecimiento cúbico $\Theta(N^3)$ del enfoque tradicional, pero fijó un umbral claro ($N \geq 256$): bajo este tamaño, el costo de particionar y alojar submatrices supera el ahorro de multiplicaciones. En consecuencia, elegir un algoritmo óptimo exige sopesar no solo su cota asintótica, sino también las propiedades de la entrada y el contexto práctico de ejecución.

Referencias

- [1] ByteQuest. *Strassen Algorithm Visually Explained*. <https://www.youtube.com/watch?v=2IgZuVGwEb0>. 2023.
- [2] Thomas H. Cormen et al. *Introduction to Algorithms*. 3rd. MIT Press, 2009.
- [3] Educative Answers Team. *Hoare's vs. Lomuto partition scheme in QuickSort*. <https://www.educative.io/answers/hoares-vs-lomuto-partition-scheme-in-quicksort>. 2024.

- [4] David Hales. «An Introduction to Replicability in Computer Science Research». En: *Journal of Computational Methods* 13.1 (2017), págs. 123-145.
- [5] Donald E. Knuth. *The Art of Computer Programming, Volume 3: (2nd ed.) Sorting and Searching*. USA: Addison Wesley Longman, 1998.
- [6] Stable Sort. *Longest Increasing Subsequence $O(n \log n)$ dynamic programming Java source code*. <https://www.youtube.com/watch?v=22s1xxRvy28>. 2019.